Research Multi Tenancy

Research

Bas Smit

Fontys University of Applied Sciences

Fontys
University of Applied Sciences

Contents

1 Introduction **1**

2 Multi-Tenancy Strategies **2**
2.1 Shared Application — Shared Database 2
2.2 Shared Application — Separate Database per Tenant 2
2.3 Shared Application — Separate Schema per Tenant 2
2.4 Separate Application Instance — Shared Infrastructure 2
2.5 Dedicated Infrastructure per Tenant 3
2.6 Hybrid Multi-Tenancy 3

3 Application to the SaZ Platform **4**
3.1 Current Architecture 4
3.2 Recommended Approach 4
3.3 Tenant Routing 4
3.4 Keycloak Realm Per Tenant 5
3.5 Onboarding Automation 5
3.6 Trade-offs and Risks 5

1 Introduction

Samen aan Z (SaZ) is a Kubernetes-based microservices platform currently deployed for a single customer: the Province of Antwerp. The platform consists of several independently deployed services across multiple Kubernetes namespaces (`data-api`, `survey`, `dashboard`), backed by dedicated PostgreSQL databases per domain, a MongoDB instance, a RabbitMQ message broker, and Keycloak for authentication. All services are containerised and deployed via Helm charts.

At present, the platform is strictly single-tenant — one deployment serves one organisation. As the platform matures, there is a realistic need to onboard additional customers without running a completely separate deployment per customer. This research investigates what multi-tenancy means in the context of the SaZ infrastructure, what strategies are available, and which approach is most suitable given the existing architecture.

2

Multi-Tenancy Strategies

Multi-tenancy can be implemented at different levels of the stack. The six strategies below range from fully shared to fully dedicated, each with different trade-offs in isolation, resource cost, and implementation effort.

2.1. Shared Application — Shared Database

A single application instance serves all tenants, and all tenant data lives in the same database. Every table includes a `tenant_id` column and all queries are filtered by it. This is the most resource-efficient model and the simplest to operate, but it places the entire burden of isolation on the application layer. A missing `WHERE tenant_id = ?` clause is enough to expose one tenant's data to another, making this the highest-risk option. It is generally only appropriate when tenants are loosely isolated by design (e.g. internal teams within one organisation).

2.2. Shared Application — Separate Database per Tenant

The application is shared, but each tenant gets its own database instance. A tenant identifier (extracted from the request context, subdomain, or JWT claim) is used to select the correct database connection at runtime. Data isolation is strong — a query against one tenant's database physically cannot touch another's — and backups and restores can be performed per tenant. The trade-off is connection pool overhead: with many tenants, maintaining open pools to dozens of databases becomes expensive.

2.3. Shared Application — Separate Schema per Tenant

A middle ground between the two approaches above. All tenants share a single database server, but each tenant has its own schema (a named namespace within the database). The application switches the active schema based on the tenant context before executing queries. This avoids the connection-pool pressure of separate databases while still providing meaningful data separation — a query in one schema cannot accidentally read rows from another. Schema-level access controls in PostgreSQL reinforce this boundary. The main drawback is that schema migrations must be applied to every tenant schema, which requires a coordinated migration strategy.

2.4. Separate Application Instance — Shared Infrastructure

Each tenant runs its own deployed instance of the application (separate pods, services, and databases) but shares the underlying cluster infrastructure: load balancer, ingress controller, logging stack, CI/CD pipelines, and container registry. This is the model recommended for SaZ in Chapter 3. Isolation is high and tenants are independently deployable and upgradeable. The cost is proportional resource usage — each tenant's instance consumes CPU and memory — but shared infrastructure keeps the operational overhead manageable.

2.5. Dedicated Infrastructure per Tenant

Every tenant receives a fully independent environment: separate Kubernetes cluster (or cloud subscription), separate networking, and separate infrastructure components. This maximises isolation and allows per-tenant SLA agreements, compliance boundaries, and network policies. It is the appropriate choice for highly regulated industries or customers with contractual data residency requirements. Operationally it is the most expensive model, as infrastructure costs and management effort scale linearly with the number of tenants.

2.6. Hybrid Multi-Tenancy

In practice, many platforms combine models. For example: standard customers share an application instance with schema-level database isolation, while enterprise or regulated customers receive a dedicated application instance on shared infrastructure, and a small number of highly sensitive customers get fully dedicated infrastructure. This tiered approach balances cost efficiency for the majority with the ability to meet stricter requirements for customers who need them. The trade-off is increased architectural complexity — the platform must support multiple isolation levels simultaneously, which complicates testing, deployment pipelines, and support processes.

3

Application to the SaZ Platform

3.1. Current Architecture

The SaZ platform already applies several patterns that form a natural foundation for multi-tenancy:

- **Namespace isolation** — services are already split across `data-api`, `survey`, and `dashboard` namespaces, making namespace-per-tenant a low-friction extension.
- **Database-per-service** — each domain already has its own PostgreSQL database (`surveydb`, `responsedb`, `userdb`, etc.), so extending this to database-per-tenant per service is architecturally consistent.
- **Keycloak SSO** — Keycloak supports multiple *realms* natively, where each realm can represent a tenant with its own users, roles, and client configuration.
- **Helm charts** — all services are templated via Helm, meaning a new tenant deployment can be parameterised and automated rather than hand-crafted.

3.2. Strategy Assessment for SaZ

Each strategy from Chapter 2 is evaluated below against the SaZ architecture.

Shared Application — Shared Database is not suitable. SaZ is built as a microservices platform with multiple separate databases per domain. Collapsing all tenant data into shared tables would require adding `tenant_id` columns and query filters throughout every service, representing significant rework with high risk of data leaks.

Shared Application — Separate Database per Tenant is technically feasible but introduces tenant-aware connection routing into every service. Each microservice would need logic to resolve the correct database connection from the request context, which is a cross-cutting change across the entire codebase.

Shared Application — Separate Schema per Tenant is a lighter version of the above and could be applied to the PostgreSQL instances without major code changes using PostgreSQL's `search_path`. However, coordinating schema migrations across all tenant schemas adds operational risk, and it still requires application-level changes to switch schema context per request.

Separate Application Instance — Shared Infrastructure is the recommended approach for SaZ (see Section **??**). It requires no application code changes, fits the existing Helm-based deployment model, and provides strong data isolation by default.

Dedicated Infrastructure per Tenant would mean running a separate Kubernetes cluster per customer. This is prohibitively expensive for SaZ at this stage and eliminates the operational benefit of a shared cluster. It would only be justified if a customer had strict regulatory data residency requirements.

Hybrid Multi-Tenancy is a viable future evolution. Once the platform matures, standard customers could be moved to a shared-schema model to reduce resource usage, while enterprise customers retain

dedicated namespaces. However, for an initial implementation the added complexity is not warranted.

3.3. Recommended Approach

Given the existing architecture, the most practical approach is **namespace-per-tenant with shared infrastructure components**. Concretely:

1. Each tenant receives its own Kubernetes namespace containing the application services and databases, deployed via the existing Helm charts with tenant-specific values.
2. Shared infrastructure — RabbitMQ, Seq (logging), the Docker registry, cert-manager, and MetalLB — remains cluster-wide and is shared across tenants.
3. Keycloak is configured with a dedicated realm per tenant, keeping user stores and authentication flows fully isolated.
4. A new tenant onboarding process is scripted (extending the existing `platformdeploy` scripts) to provision the namespace, apply Helm charts, create the Keycloak realm, and configure ingress rules automatically.

This approach minimises changes to the application code (no `tenant_id` plumbing required), preserves strong data isolation, and keeps operational overhead manageable by reusing the existing Helm-based deployment pipeline.

3.4. Tenant Routing

With namespace-per-tenant, every tenant needs its own entry point into the cluster. The cleanest approach for SaZ is **subdomain-based routing**: each tenant gets a subdomain such as `tenant-a.saz.com`, and the Nginx Ingress Controller resolves the hostname to the correct namespace's services.

The existing ingress manifests (`sazazewayingress.yaml`, `surveyingress.yaml`, etc.) are already templated via Helm. To support multiple tenants, the `host` field in each ingress resource becomes a Helm value:

```
# values.yaml (per-tenant)
ingress:
  host: tenant-a.saz.com
```

Cert-manager's ClusterIssuer already handles TLS automatically via ACME, so each new subdomain gets its own certificate without manual intervention.

Within the cluster, services communicate via Kubernetes DNS (`service.namespace.svc`), so internal traffic is already namespace-scoped. No application code changes are needed for internal routing — only the ingress layer is tenant-aware.

3.5. Keycloak Realm Per Tenant

Keycloak's *realm* concept maps directly onto a tenant. Each realm is a fully independent authentication domain with its own users, groups, roles, and OAuth2 clients. The SaZ services authenticate via the `sazkeycloak` deployment, which needs one realm per tenant.

In practice this means:

- Each realm has its own set of OAuth2 clients registered for the services in that tenant's namespace (e.g. `sazazeway-client`, `survey-client`).
- The `issuer` URL in a JWT token includes the realm name (`/realms/tenant-a`), so services can validate that a token belongs to the correct tenant by checking the issuer claim.
- Services receive the Keycloak realm URL as an environment variable via their Helm values file, meaning no code changes are needed — only configuration differs between tenants.

Realm creation can be automated via the Keycloak REST API or by providing a realm export JSON at startup. The latter is preferable for reproducibility: a base realm template is exported once and used as the seed for every new tenant, ensuring consistent role and client configuration across deployments.

3.6. Onboarding Automation

The existing `platformdeploy` scripts already orchestrate a full deployment. Tenant onboarding is an extension of this: a new script (e.g. `tenantprovision.ps1`) accepts a tenant name and domain as parameters and runs the following steps in order:

1. **Create namespace** — `kubectl create namespace $tenantName`
2. **Apply NetworkPolicy** — lock down ingress/egress so the namespace cannot communicate with other tenant namespaces.
3. **Deploy services** — run `helm install` for each chart in `infrastructure/data-api`, `infrastructure/survey`, and `infrastructure/dashboard`, passing a values file generated from the tenant name and domain.
4. **Provision Keycloak realm** — call the Keycloak Admin REST API to import the base realm template with the tenant name substituted in, or apply a pre-rendered realm export JSON.
5. **Apply ingress rules** — deploy the tenant-specific ingress resources with the correct `host` values so Nginx routes traffic to the new namespace.

Teardown follows the same steps in reverse: delete the ingress, remove the Keycloak realm, then delete the namespace (which cascades to all resources inside it).

3.7. Trade-offs and Risks

The main cost of this approach is resource consumption: each tenant's namespace runs its own database pods and service instances. For a small number of tenants this is acceptable; at larger scale a schema-per-tenant model within shared database instances would reduce resource usage at the cost of more complex data routing logic. Security boundaries between namespaces should be enforced with Kubernetes `NetworkPolicy` resources to prevent unintended cross-tenant traffic on the shared cluster.

4

Conclusion

Of the six multi-tenancy strategies evaluated, **Separate Application Instance with Shared Infrastructure** is the most suitable for the SaZ platform in its current state. It is the only approach that requires no changes to application code, aligns naturally with the existing Helm-based deployment pipeline, and provides strong tenant isolation by default through Kubernetes namespace boundaries and separate database instances per tenant.

The other strategies were ruled out for the following reasons: shared-database models (shared schema, separate schema, or row-level) all require cross-cutting code changes and carry a higher risk of data leaks; dedicated infrastructure per tenant is operationally too expensive; and hybrid multi-tenancy adds unnecessary complexity at this stage of the platform's maturity.

The practical implementation path is clear: parameterise the existing Helm charts with tenant-specific values, route tenant traffic via subdomains at the Nginx ingress layer, and provision a Keycloak realm per tenant for authentication isolation. Onboarding a new tenant becomes a scripted, repeatable process that extends the existing `platformdeploy` tooling. As the platform scales and the number of tenants grows, a migration toward a hybrid model — where lower-tier tenants share application instances with schema-level database isolation — can reduce resource consumption without requiring a full architectural redesign.